# Priority Token Insertion

Category B Coding Assignment - Problem 1

## Problem summary

Implement a custom singly linked list with addFirst, addLast, and insertAt(index, token). Index is zero-based from 0 through the current size. Do not use java.util.LinkedList.

## Java solution

```
public class Main {
 static class Node { int data; Node next; Node(int d){data=d;} }
 static class TokenList {
  private Node head;
  void addFirst(int token) { Node n=new Node(token); n.next=head; head=n; }
  void addLast(int token) { Node n=new Node(token); if(head==null){head=n;return;} Node p=head; while(p.next!=nu
  void insertAt(int index,int token) {
   if(index<0)throw new IndexOutOfBoundsException("index");
   if(index==0){addFirst(token);return;}
   Node p=head; for(int i=0;i<index-1 && p!=null;i++)p=p.next;
   if(p==null)throw new IndexOutOfBoundsException("index");
   Node n=new Node(token); n.next=p.next; p.next=n;
  }
  void print(){Node p=head;while(p!=null){System.out.print(p.data+" -> ");p=p.next;}System.out.println("null");}
 }
 public static void main(String[] args){TokenList l=new TokenList();l.addLast(101);l.addLast(102);l.addLast(103)
}
```

## Sample output

```
100 -> 101 -> 150 -> 102 -> 103 -> 104 -> null
```

## Complexity

addFirst: O(1). addLast: O(n) with a head-only list. insertAt: O(n) traversal and O(1) pointer rewiring; each insertion uses O(1) extra space.